

Coupled Energy-Minimization Smoothing and Topological Mesh Adaptation: Design of an Adaptivity Loop for Norma

Alejandro Mota

August 2026

Abstract

Energy-minimization smoothing (EMS) improves a mesh by minimizing a hyperelastic distortion energy over the nodal positions, with each element measured against an ideal reference element. Smoothing alone, however, terminates at configurations where no continuous motion of the nodes can further reduce the energy, while the mesh remains poor because the obstruction is topological. Numerical experiments show that alternating EMS with targeted topological operations—edge swaps, collapses, and splits applied to the elements of highest distortion energy—improves mesh quality far beyond what either mechanism achieves in isolation. This note designs such a coupled loop for Norma, informed by a detailed reading of the adaptation logic of the Omega.h library. Three questions are addressed with concrete algorithms: when to trigger topological operations, how to guarantee a monotonic decrease of the total distortion energy across both smoothing and topological phases, and how to detect stagnation of the smoothing phase with a scale-invariant criterion. The central design decision is to replace the current history-dependent smoothing objective by an intrinsic distortion functional, well defined for any mesh topology, which then serves as a common Lyapunov function for the entire loop. The note closes with a step-by-step implementation plan, recommendations for topology and adjacency data structures, and a ranked list of risks.

Contents

1	Motivation and observed workflow	2
2	Existing infrastructure in Norma	3
2.1	EMS as a solid-mechanics solve	3
2.2	Assets already in place	3
2.3	Gaps	4
3	The adaptation logic of Omega.h	4
3.1	Two tiers of thresholds	4
3.2	Control flow	4
3.3	Termination	5
4	An intrinsic distortion functional	5
4.1	The obstruction	5
4.2	The functional	5
4.3	Re-baselining	6

5	Question 1: adaptivity triggers	6
6	Question 2: energy monotonicity	7
6.1	The acceptance gate	7
6.2	The smoothing phase	7
7	Question 3: stagnation detection	8
8	Architectural plan	9
9	Data structures and design patterns	10
10	Risks and open questions	11
11	Summary	11

1 Motivation and observed workflow

Mesh smoothing by energy minimization treats the mesh as a hyperelastic body whose per-element reference configuration is an ideal element, and minimizes the resulting stored energy over the nodal positions. The energy acts as a barrier against element inversion and drives every element toward the ideal shape. The mechanism is purely geometric: the connectivity is fixed, and only the nodal positions move.

This fixity is also the limitation. Many poor meshes contain configurations—slivers wedged between well-shaped neighbors, vertices of excessive valence, edges shared by too many or too few elements—whose distortion cannot be removed by any continuous motion of the nodes. Smoothing then stalls: the energy plateaus, or the line search fails against the inversion barrier while the gradient remains large. The obstruction is topological, and the remedy is a local change of connectivity. Conversely, topological operations applied to an unsmoothed mesh are evaluated at poor nodal positions and are frequently rejected or mis-ranked. The two mechanisms are complementary, and the literature on tetrahedral mesh improvement confirms that their combination is substantially stronger than either alone [3, 4].

The workflow this note develops is therefore iterative:

1. run EMS until the energy decrease stagnates;
2. identify the elements of highest distortion energy;
3. apply topological operations to the neighborhoods of those elements, accepting only operations that decrease the energy;
4. resume EMS, which the changed topology now permits to relax further;
5. repeat until neither mechanism makes progress.

The design questions are precisely where the coupling is delicate: the trigger for step 3, the acceptance criterion that keeps the total energy monotone across steps 1–4, and the stagnation test in step 1. Sections 5–7 address them in turn, after two preparatory sections: an inventory of the machinery Norma already possesses (Section 2) and a summary of the decision logic of Omega.h as implemented in its source code (Section 3).

2 Existing infrastructure in Norma

2.1 EMS as a solid-mechanics solve

Norma implements EMS not as a separate module but as a mode of the ordinary quasi-static `SolidMechanics` solve. A single substitution in the element loop (`src/model.jl`) replaces each element’s reference configuration by an ideal regular tetrahedron before computing the deformation gradient; everything else—assembly, solvers, boundary conditions, output—is the standard machinery. The energy is the Seth–Hill family with exponents $m = n = 2$, which diverges both as the element Jacobian tends to zero and as it tends to infinity, and therefore acts as an inversion barrier. The nodal positions are the unknowns; the minimization runs under the quasi-static integrator as pseudo-time continuation, with matrix-free steepest-descent or L-BFGS steps.

The ideal reference element is built by `create_smooth_reference` (restricted at present to four-node tetrahedra) as a regular tetrahedron of edge length h , where h follows one of four rules: equal volume to the original element, average edge length of the original element, the maximum of these two, or a user-supplied size field $s(t, x, y, z)$. In all four rules the input geometry is the *original* mesh as read from disk; the consequences of this choice for adaptivity are the subject of Section 4.

Boundary nodes move but remain on the boundary, through either axis-aligned Dirichlet conditions or the `Surface` boundary condition, which represents a boundary surface as an analytic level set $g(\mathbf{x}) = 0$ and enforces it by projection: nodes are returned to the surface before assembly (`return_to_surface!`) and the gradient is projected onto the tangent space afterward. This machinery is directly reusable for placing and constraining nodes during topological operations on curved boundaries.

2.2 Assets already in place

Three existing components carry much of the weight of the proposed loop.

1. *A per-element distortion field.* The integrated element energy is stored per block and element in `model.stored_energy`, updated on every evaluation of the model and already written to the Exodus output as an element variable. Step 2 of the workflow—finding the most distorted elements—requires no new computation.
2. *Cross-mesh state transfer.* The module `src/field_transfer.jl` performs a consistent L^2 transfer of nodal fields between meshes of the same geometry, and composes it with the variational recovery of [6] to remap integration-point internal variables. This is the machinery a future in-simulation adaptivity capability requires, and it connects to the constrained projection of the companion note on internal-variable recovery in admissible spaces (`docs/notes/internal-variable-projection`).
3. *Mid-run mesh replacement.* The module `src/swap.jl` already replaces the mesh of a running simulation: it rebuilds the model, integrator, and solver, and transfers the state to the new mesh. This is the orchestration pattern a topology change must follow, for a structural reason: `initialize_storage` aliases the integrator and solver arrays into the model matrices through raw pointers (`unsafe_wrap`), so any reallocation of the model arrays invalidates the integrator and solver. A topology change must therefore proceed by full rebuild, never by in-place resizing.

2.3 Gaps

Equally important is what does not exist. The mesh is never held in memory as a mutable object: the `ExodusDatabase` handle is the source of truth, connectivity is re-read from the file on every access, and no function writes coordinates or connectivity back. There are no adjacency structures of any kind—no node-to-element, element-to-element, or edge data—so there is at present no way to enumerate the cavity around an edge. There is no element quality module; the de facto quality measure of the code base is the smoothing energy itself. Finally, point location for state transfer is a linear scan over the elements of a block. These gaps define Steps 3 and 8 of the implementation plan in Section 8.

3 The adaptation logic of `Omega.h`

`Omega.h` [1, 2] is a metric-driven simplex-adaptation library whose decision logic is compact and, as verified directly against its source code, organized around three ideas: a two-tier system of thresholds, cavity operators filtered by acceptance criteria, and progress-based termination. This section records the logic precisely, because Section 5 transplants it.

3.1 Two tiers of thresholds

The options structure `AdaptOpts` carries *desired* and *allowed* values with distinct roles: desired values select candidates for work, allowed values are hard gates that no accepted operation may violate. The defaults are: metric edge lengths desired within $[1/\sqrt{2}, \sqrt{2}]$, length allowed up to $2\sqrt{2}$; element quality desired at least 0.30 (three dimensions), quality allowed at least 0.20. There is no lower allowed length: short edges are only ever “desired” away. The quality measure is the metric-conditioned mean ratio, an intrinsic shape measure equal to one for a metric-equilateral simplex and non-positive for a degenerate or inverted one.

3.2 Control flow

The top-level `adapt()` first evaluates an “already satisfactory” predicate—minimum quality at least the desired quality *and* all metric edge lengths within the desired band—and returns without touching the mesh if it holds. Otherwise it runs two phases.

1. *Length phase.* Alternate refinement of edges longer than $\sqrt{2}$ and collapse of edges shorter than $1/\sqrt{2}$, repeating until neither operator changes anything. Acceptance in this phase enforces only the hard gates: a refinement is accepted if the worst new element stays above the allowed quality; a collapse additionally may not create edges beyond the allowed length. Quality may therefore *decrease* in this phase, down to the floor.
2. *Quality phase.* While the minimum quality is below the desired value: attempt edge swaps; only if a full swap pass changes nothing, attempt sliver collapses; if both fail in the same pass, report failure and stop. Acceptance in this phase is strict improvement: a swap or sliver collapse is kept only if the minimum quality of the new cavity *strictly exceeds* that of the old cavity. This strict filter is the only monotonicity guarantee in the library.

Candidates for the quality phase are the elements below desired quality *dilated by four layers of adjacent elements*, because the operation that repairs a bad element frequently lies on a neighboring entity. Boundary edges are never swapped. Conflicts between overlapping cavities are resolved by a

Luby-style independent set in which the cavity of higher quality wins, with a deterministic global-identifier tiebreak; candidates not selected simply wait for the next pass.

3.3 Termination

There is no iteration counter anywhere in `adapt()`. Every operator returns false exactly when no candidate survives its filter chain, and each phase loop exits when its operators return false. Termination is thus purely progress-based: the loop runs precisely as long as some operation can be accepted. When the quality goal is unreachable, the library says so and returns the mesh in its best achieved state rather than looping.

4 An intrinsic distortion functional

4.1 The obstruction

The design problem that governs the entire coupling is that Norma's current smoothing objective is tied to the original mesh: the ideal reference of each element is built from the geometry of that element in the mesh as read from disk. After a topological operation, a new element has no original counterpart, so the objective is undefined on it; and even where defined, an energy computed against history-dependent targets cannot be compared across a topology change, because the comparison would be between different functionals. A monotonicity guarantee requires a single functional, defined for every admissible pair of topology and nodal positions, that both phases of the loop demonstrably decrease.

4.2 The functional

Let $\mathcal{T}$ denote the mesh topology and $\mathbf{x}$ the nodal positions. Define

$$D(\mathcal{T}, \mathbf{x}) = \sum_{e \in \mathcal{T}} W_e(\mathbf{x}), \quad W_e(\mathbf{x}) = \int_{\Omega_e^{\text{ideal}}} \psi(\mathbf{F}_e) dV, \quad (1)$$

where ψ is the Seth–Hill energy density already in use, $\mathbf{F}_e$ is the deformation gradient of element e measured from its ideal reference, and—this is the decision—the ideal reference of each element is the regular tetrahedron of *equal volume to that element's own current volume*. This is the existing "equal volume" rule of `create_smooth_reference`, applied to the current rather than the original geometry.

With this choice the energy of an element depends only on its own present shape: it is invariant under uniform scaling, vanishes exactly for a regular tetrahedron, and diverges at degeneracy. The functional (1) is therefore *intrinsic*—a function of $(\mathcal{T}, \mathbf{x})$ alone, with no dependence on mesh history—and is well defined for any tetrahedral mesh whatsoever. It belongs to the same class as the mean-ratio quality of `Omega_h`, which is likewise a scale-invariant shape measure of the current element; the difference is that (1) is an energy with an inversion barrier rather than a bounded quality, which is precisely what makes it minimizable by the existing solvers. For threshold purposes define the normalized density

$$\hat{w}_e = \frac{W_e}{V_e}, \quad (2)$$

with V_e the current element volume, so that $\hat{w}_e = 0$ for an ideal element and $\hat{w}_e \rightarrow \infty$ at degeneracy. Note the orientation: minimum quality in `Omega_h` corresponds to *maximum* energy density here.

Remark 1. *Size-targeting is deliberately excluded from (1). A target size field changes what the mesh should be, and operations that pursue it—refinement toward a smaller h , coarsening toward a larger one—intentionally change the objective and cannot be monotone in any fixed functional. This mirrors the phase separation of Omega_h: its length phase enforces only hard floors, and its strict improvement filter lives exclusively in the quality phase. The proposed loop keeps the same separation: an optional size phase driven by the existing size-field machinery, gated only by the hard energy ceiling, followed by the quality phase to which the monotonicity guarantee of Section 6 applies.*

4.3 Re-baselining

The smoothing solver requires a *fixed* objective during a solve: if the ideal reference tracked the current volume continuously, the target would move with the unknowns. The resolution is re-baselining at phase boundaries: at the start of each smoothing phase, set the smoothing reference configuration to the current coordinates and build the equal-volume ideal references from it. The phase objective then coincides with $D(\mathcal{T}, \cdot)$ at the start of the phase and remains a fixed, minimizable functional throughout it. The relation between the frozen-target phase objective and the intrinsic functional is that of a majorize–minimize surrogate; the safeguard of Proposition 1 converts the practical descent of D into a guaranteed one.

5 Question 1: adaptivity triggers

The translation of the Omega_h logic into the energy setting is summarized in Table 1 and specified below.

Omega_h	Proposed EMS loop
mean-ratio quality (max. 1, degenerate ≤ 0)	energy density $\hat{w}_e$ (min. 0, degenerate $\rightarrow \infty$)
<code>min_quality_desired</code> (candidate selection)	$\hat{w}_{\text{des}}$: element is worth working on
<code>min_quality_allowed</code> (hard floor)	$\hat{w}_{\text{all}} > \hat{w}_{\text{des}}$: hard ceiling
metric length band $[1/\sqrt{2}, \sqrt{2}]$	optional size phase against size field $h(\mathbf{X})$
<code>nsilver_layers</code> = 4 adjacency layers	same: dilate candidate set by n_{layers} rings
swap first, sliver-collapse as fallback	same order
strict cavity-quality improvement filter	strict cavity-energy decrease (Section 6)
progress-based termination, no counters	same

Table 1: Correspondence between the Omega_h adaptation logic and its proposed energy-based analogue. The orientation inverts: minimum quality corresponds to maximum energy density.

1. *Trigger for the topology phase.* Enter the topology phase when the smoothing phase has stagnated in the sense of Section 7 and $\max_e \hat{w}_e > \hat{w}_{\text{des}}$. This is the “already satisfactory” predicate of Omega_h with the length clause removed (no size targeting) and quality replaced by energy density.
2. *Candidate set.* All elements with $\hat{w}_e > \hat{w}_{\text{des}}$, dilated by n_{layers} rings of element adjacency (default four, following Omega_h); the operation candidates are the interior edges of this set. Boundary edges are excluded from swapping; they participate in collapses and splits only under the surface constraints of Section 8.

3. *Order within a pass.* Swaps first; collapses only when a full swap pass accepts nothing. Swaps move no nodes and therefore compose trivially with smoothing; collapses remove resolution and are the fallback. Splits participate where a bisection lowers the cavity energy (Remark 2).
4. *Outer termination.* The coupled loop ends when the smoothing phase reports stagnation or convergence and the subsequent topology phase accepts zero operations, or when smoothing converges with $\max_e \hat{w}_e \leq \hat{w}_{\text{des}}$. Termination is progress-based, as in Omega.h; a large safety cap on outer iterations is retained for defense but is not the semantic terminator. When the goal is unreachable, the loop reports it and returns the best achieved mesh.

6 Question 2: energy monotonicity

The reading of the Omega.h source localizes its monotonicity exactly: strict improvement is guaranteed only where the strict filter is applied—swaps and sliver collapses—while refinement and size-driven collapse may degrade quality to the floor. Adapting the criterion to the EMS framework therefore means applying a strict-decrease filter, stated in energy, to *every* operation of the quality phase, and pairing it with a monotone smoothing phase. Both are stated against the intrinsic functional (1).

6.1 The acceptance gate

A topological operation replaces the elements C_{old} of a cavity by new elements C_{new} at fixed nodal positions $\mathbf{x}$ (for a split, the new node is placed and locally relaxed first; see Remark 3). The operation is accepted if and only if

$$\sum_{e \in C_{\text{new}}} W_e(\mathbf{x}) \leq \sum_{e \in C_{\text{old}}} W_e(\mathbf{x}) - \delta, \quad \text{and} \quad \max_{e \in C_{\text{new}}} \hat{w}_e \leq \hat{w}_{\text{all}}, \quad (3)$$

with $\delta > 0$ a small relative margin (for example 10^{-8} times the old cavity energy) that precludes cycles of operations accepted at zero gain. The cavity *sum* replaces the cavity minimum of Omega.h deliberately: the sum is the restriction of the global functional to the cavity, so local acceptance implies global descent, which is the property the loop needs. The ceiling $\hat{w}_{\text{all}}$ plays the role of the allowed-quality floor; element inversion is excluded automatically because the Seth–Hill barrier sends $\hat{w}_e$ to infinity there.

6.2 The smoothing phase

The L-BFGS path of the smoothing solver already performs an Armijo backtracking line search on the energy, so each smoothing phase is monotone in its own frozen-target objective; the steepest-descent path currently backtracks on the residual norm instead and should use the energy merit for this loop. With re-baselining at phase start (Section 4), the phase objective agrees with $D(\mathcal{T}, \cdot)$ at the initial iterate. Because the phase objective holds its targets fixed while D is intrinsic, the phase is completed by a safeguard: evaluate D once at phase end, and roll the phase back if D increased. The safeguard costs one evaluation and is expected to be inert in practice; its purpose is to turn practical descent into provable descent.

Proposition 1 (Global descent and finite termination of the topological work). *Let the coupled loop consist of smoothing phases with the end-of-phase safeguard and topological operations accepted by the gate (3). Then the sequence of values of D sampled at phase boundaries and after each accepted*

operation is nonincreasing and bounded below by zero, hence convergent; each accepted operation decreases D by at least δ times the corresponding cavity energy at acceptance, so if δ is enforced against a fixed positive reference energy the number of accepted topological operations is finite.

Proof. Only cavity elements change in a topological operation, so (3) implies D decreases by at least δ across it. The safeguard makes each smoothing phase nonincreasing in D by construction. A nonincreasing sequence bounded below converges, and a uniform positive decrement per accepted operation bounds their number. $\square$

The guarantee lives in exactly one place: a single acceptance routine implementing (3), called by every operator. No operator carries its own criterion.

Remark 2 (Splits participate honestly). *Because the equal-volume energy is scale-invariant, bisecting a badly stretched element into two better-shaped children genuinely lowers the cavity energy, and refinement needs no exemption from the gate (3). Only size-driven refinement toward a target $h(\mathbf{X})$ is exempt, in the separate size phase, where it is gated by the ceiling alone.*

Remark 3 (Local relaxation before the test). *For a split, place the new node at the edge midpoint—projected to the surface by `return_to_surface!` when the edge lies on the boundary—and run a few iterations of cavity-local smoothing with the cavity boundary fixed before evaluating (3). The cost is a handful of element evaluations; the benefit is that each operation is judged at its locally relaxed optimum rather than at an arbitrary initial placement, which raises the acceptance rate substantially. This is the natural advantage of an energy framework over the position-free acceptance tests of Ω_h , and it is the precise mechanism by which the alternation outperforms either method alone: each accepted operation unlocks relaxation that the previous topology forbade.*

7 Question 3: stagnation detection

The smoothing solver currently stops on a gradient-norm test with absolute and relative tolerances—a *convergence* criterion. Stagnation, the absence of significant further decrease, requires a criterion that is scale-free, tolerant of plateaus, and inexpensive. The smoothing phase should end at the first of the following three events, and report which one occurred.

1. *Windowed relative decrease.* Let W_n denote the phase objective at iteration n and let m be a window length (default $m = 10$). Stop when

$$W_{n-m} - W_n \leq \tau_{\text{stag}} (W_0 - W_n), \quad \tau_{\text{stag}} = 10^{-3} \text{ by default.} \quad (4)$$

In words: the decrease achieved over the last m iterations is a negligible fraction of the decrease achieved since the phase began. The test is invariant under affine rescaling of the energy and under problem size, and the window absorbs the short plateaus that quasi-Newton iterations produce near barrier-dominated configurations. Since Armijo backtracking makes $\{W_n\}$ monotone, the left side is nonnegative and the test is well posed.

2. *Convergence proper.* The existing projected-gradient test passes: the phase ended because the mesh is smooth, not because it is obstructed.
3. *Topological lock.* The line search backtracks to failure, or the non-positive-Jacobian guard trips repeatedly, while the projected gradient remains large. This is not convergence: it is the inversion barrier certifying that no continuous nodal motion can improve the mesh from the current configuration. It is the strongest possible trigger for the topology phase and should be detected and reported explicitly.

The soundness of (4) rests on the standard descent inequality: for a continuously differentiable objective with Armijo backtracking, the per-iteration decrease is bounded below by a multiple of the squared projected-gradient norm times the accepted step [5]. Sustained failure to decrease therefore certifies either a vanishing projected gradient (event 2 in disguise) or a collapsing step length against the barrier (event 3); the windowed test detects both with a single properly scaled quantity and no problem-dependent tuning.

Distinguishing the three events gives the outer loop its terminator at no additional cost: event 2 with $\max_e \hat{w}_e \leq \hat{w}_{\text{des}}$ means the loop is finished; events 1 and 3 call for a topology phase; a topology phase that accepts nothing after event 1 or 3 means the loop has reached the best this machinery can achieve, which is reported rather than concealed—the exact analogue of the explicit failure report of `Omega.h` when its quality goal is unreachable.

8 Architectural plan

The implementation follows the repository convention of paired files: `src/adapt.types.jl` for the option and cache types, `src/adapt.jl` for the methods. The steps are ordered so that each is independently testable, with the first coupled loop appearing at Step 4.

1. *Intrinsic energy kernel.* Factor the per-element energy evaluation out of the assembly loop into a kernel callable on an arbitrary subset of elements, add the equal-volume re-baselined variant of Section 4, and register the density $\hat{w}_e$ as an Exodus element variable. Testable immediately against `model.stored.energy` on the existing `examples/ems` cases.
2. *Stagnation criterion.* Add the windowed test (4) and the lock detection to the solver stopping logic, returning a phase outcome (converged, stagnated, locked) rather than a Boolean, and switch the steepest-descent merit to the energy. Independently testable: the strongly distorted examples should report the lock outcome where they currently exhaust iterations.
3. *In-memory topology and adjacency.* A topology cache built once per topology phase from the Exodus-read connectivity: coordinates, per-block connectivity, and the three adjacencies the code base lacks—element-to-element across faces (by sorted-face matching), node-to-elements in compressed sparse row form, and an edge table mapping each edge to its incident elements. Boundary faces are identified as unpaired faces and cross-checked against the side sets; node-set and side-set membership is stored as per-entity flags so operations can update it. Four-node tetrahedra only, matching the existing restriction of `create.smooth.reference`.
4. *Edge swap and the acceptance gate; first coupled loop.* Implement the three-dimensional edge swap: extract the loop of elements around an edge (at most seven, following `Omega.h`), enumerate the tabulated triangulations of the surrounding polygon, and select the best; implement the single acceptance routine (3); wire the outer driver (smoothing phase, candidate collection, swap passes, repeat). Swaps move no nodes and need no state transfer, so this milestone alone should reproduce most of the observed benefit of the alternation.
5. *Edge collapse.* Sliver removal under the gate (3); collapses on the boundary are restricted to boundary edges with the surviving node kept on the surface through the `Surface` level-set machinery; nodes belonging to node sets are never removed; an orientation guard rejects collapses that would invert any element (already implied by the ceiling, but checked cheaply before the energy evaluation).

6. *Edge split and the optional size phase.* Midpoint placement with surface projection for boundary edges, cavity-local relaxation per Remark 3, then the gate. Optionally, the size phase: refine edges longer than $\sqrt{2}h(\mathbf{X})$ and collapse edges shorter than $h(\mathbf{X})/\sqrt{2}$ against the existing size-field pipeline, with h carried as a nodal field whose value at a new node is the mean of its parents' values; this phase is gated by the ceiling only and sits outside Proposition 1, by the phase separation of Section 4.
7. *Orchestration and output.* After each topology phase, rebuild the model, integrator, and solver through the existing mesh-replacement pattern of `src/swap.jl`, which avoids the pointer-aliasing hazard by construction. Write each adapted mesh as a fresh Exodus file in a numbered sequence: the Exodus format cannot change element counts within a file, so per-phase files are the practical route, and the replacement machinery already validates it. Input schema: an `adaptivity` block carrying $\hat{w}_{\text{des}}$, $\hat{w}_{\text{all}}$, the stagnation window and tolerance, the number of adjacency layers, and per-operator switches, mirroring `AdaptOpts`.
8. *In-simulation adaptivity (later).* Invoke the driver from the evolution loop behind a criterion in the style of the existing mesh-swap criteria, and transfer the state through `field_transfer.jl`. Two prerequisites are known in advance: point location must be accelerated from the linear scan to a spatial hash or bounding-volume hierarchy before per-adaptation transfer is affordable, and the transfer of internal variables is exactly where the constrained projection of the companion note enters, so that norm-like variables such as the equivalent plastic strain remain admissible under the L^2 transfer.

9 Data structures and design patterns

1. *Rebuild per phase, not incremental maintenance.* `Omega_h` rebuilds its topology arrays wholesale after each operator pass rather than patching them. The same pattern is right here: topology phases are rare relative to smoothing iterations, wholesale rebuild is simple and cache-friendly, and it eliminates the entire class of stale-adjacency defects. Within a pass, operations are applied through tombstones—dead elements and nodes are marked, new ones appended—and the arrays are compacted once at the end of the pass.
2. *One cavity abstraction, one gate.* Every operator produces a cavity proposal: the identifiers of the old elements, the proposed new connectivity, and optionally a new node. A single routine evaluates (3) and applies or discards the proposal. The monotonicity guarantee then has exactly one owner in the code.
3. *Sequential priority order now, independent sets later.* The Luby-style independent set of `Omega_h` exists for distributed parallelism. In serial, sorting the candidates by cavity energy in decreasing order and applying them sequentially is simpler and equally effective; a cavity that touches an element already modified in the current pass is deferred to the next pass, exactly as unselected `Omega_h` candidates wait for theirs. If the topology phase is ever threaded, adopt the independent set with the same comparator: the better cavity wins, with a deterministic identifier tiebreak.
4. *Exodus out of the hot path.* Connectivity is at present re-read from the database handle inside the assembly loop. The topology cache of Step 3 should become the in-memory source of truth during adaptation, with Exodus demoted to input and output at phase boundaries. This change benefits plain EMS performance even before any topological operation exists.

5. *Outcome types over Booleans.* The solver reports converged, stagnated, or locked; each operator pass reports its number of accepted operations; the driver’s control flow is written entirely against these outcomes. This makes the progress-based termination of Section 5 auditable in the logs, in the same way that each `Omega.h` operator returns false exactly when its filter chain leaves no candidate.

10 Risks and open questions

Ranked by importance.

1. *The re-baselining subtlety.* The intrinsic equal-volume functional of Section 4 is the load-bearing design decision. It should be validated by a small standalone prototype—a single cavity with hand-verified energies before and after a swap, a collapse, and a split—before Step 4 is undertaken.
2. *Boundary collapses on curved surfaces.* The level-set projection and the energy test can conflict: a collapse that lowers the cavity energy may move the surviving node to a surface position that raises it again after projection. The gate must therefore be evaluated *after* projection, and boundary collapses should be expected to have a lower acceptance rate.
3. *Scope restriction to four-node tetrahedra.* Inherited from `create_smooth_reference` and acceptable for the first version, but it should be stated explicitly in the input documentation.
4. *Explicit dynamics.* If the loop is later used within explicit transient simulations, edge collapses shrink the stable time step; the lower band of the size phase then ceases to be optional and must be enforced as a floor on element size.

11 Summary

The coupled loop rests on one design decision and three algorithms. The decision is to replace the history-tied smoothing objective by the intrinsic distortion functional (1), defined through equal-volume ideal elements on the current geometry, so that a single quantity is meaningful across every smoothing step and every topological operation. The algorithms are: the transplanted `Omega.h` trigger logic—two-tier thresholds on the energy density, candidate dilation by adjacency layers, swaps before collapses, progress-based termination—; the acceptance gate (3), which strengthens the strict-improvement filter of `Omega.h` from the cavity minimum to the cavity sum and thereby makes local acceptance imply global descent (Proposition 1); and the windowed stagnation test (4) with explicit detection of the topological lock, which supplies both the trigger for the topology phase and the terminator of the outer loop. Norma already provides the per-element energy field, the cross-mesh state transfer, and the mesh-replacement orchestration; the genuinely new construction is the in-memory topology cache with adjacency, the three cavity operators, and the single gate through which all of them pass.

References

- [1] D. A. Ibanez et al. `Omega.h`: reliable mesh adaptation. Sandia National Laboratories, https://github.com/sandialabs/omega_h.

- [2] D. A. Ibanez. *Conformal Mesh Adaptation on Heterogeneous Supercomputers*. PhD thesis, Rensselaer Polytechnic Institute, 2016.
- [3] L. A. Freitag, C. Ollivier-Gooch. Tetrahedral mesh improvement using swapping and smoothing. *International Journal for Numerical Methods in Engineering* 40 (1997) 3979–4002.
- [4] B. M. Klingner, J. R. Shewchuk. Aggressive tetrahedral mesh improvement. In: *Proceedings of the 16th International Meshing Roundtable*, Springer, 2007, 3–23.
- [5] J. Nocedal, S. J. Wright. *Numerical Optimization*, 2nd edition. Springer, 2006.
- [6] A. Mota, W. Sun, J. T. Ostien, J. W. Foulk III, K. N. Long. Lie-group interpolation and variational recovery for internal variables. *Computational Mechanics* 52 (2013) 1281–1299.